Лабораторна робота № 7

Мета роботи:

Сформувати практичні навички роботи з відеопотоком у ROS2, навчитися отримувати зображення з камери, виконувати базову обробку за допомогою OpenCV та інтегрувати комп'ютерний зір у робототехнічну систему.

Репозиторій з роботою: https://github.com/zakappaa/ROS2_Ginzburg

Хід роботи

ROS2-середовище активовано командами source. Змінна ROS_DISTRO має значення humble, OpenCV і NumPy доступні. На цьому етапі топіка камери в системі ще немає: ros2 topic list показує лише службові топіки.

Створення пакету Vision

У ~/ros2_ws/src створено пакет командою: ros2 pkg create --build-type ament_python --license Apache-2.0 --dependencies rclpy sensor_msgs cv_bridge --node-name vision_node vision_pkg.

Пакет має тип збірки ament_python і залежності rclpy, sensor_msgs, cv_bridge, залежність від OpenCV додано в package.xml як exec_depend (рис. 1). Другу ноду (camera_sim_node) додано вручну й зареєстровано в setup.py. Пакет зібрано командою colcon build --packages-select vision_pkg; обидві ноди доступні через ros2 pkg executables.

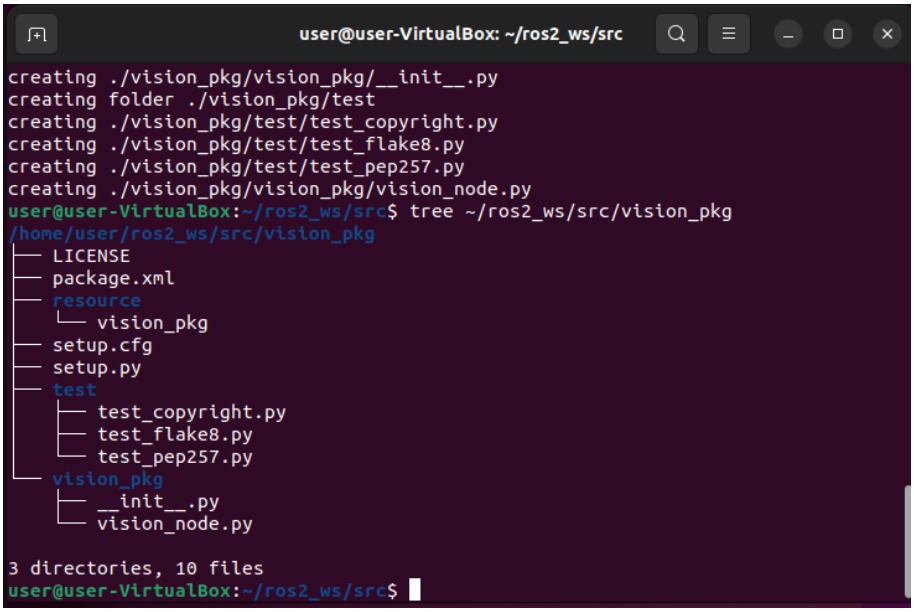


Рисунок 1 - Створення пакету vision_pkg та його структура

Джерело відео: симуляція камери

Оскільки у віртуальній машині немає фізичної камери, відеопотік імітує нода camera_sim_node. Вона публікує в топік /camera/image кадри 640×480 у кодуванні bgr8 з частотою 15 Гц. Сцена складається з нерухомих фігур на фоні з градієнтом, рухомого помаранчевого кола та гаусового шуму, який імітує шум сенсора реальної камери.

Після запуску симулятора в ros2 topic list -t з'явився топік камери /camera/image [sensor_msgs/msg/Image]. Команда ros2 topic hz показала частоту <значення average rate з рис. 6> Гц, а ros2 topic echo --no-arr --once - структуру повідомлення: height: 480, width: 640, encoding: bgr8, step: 1920, довжина data 921 600 байт (рис. 2).

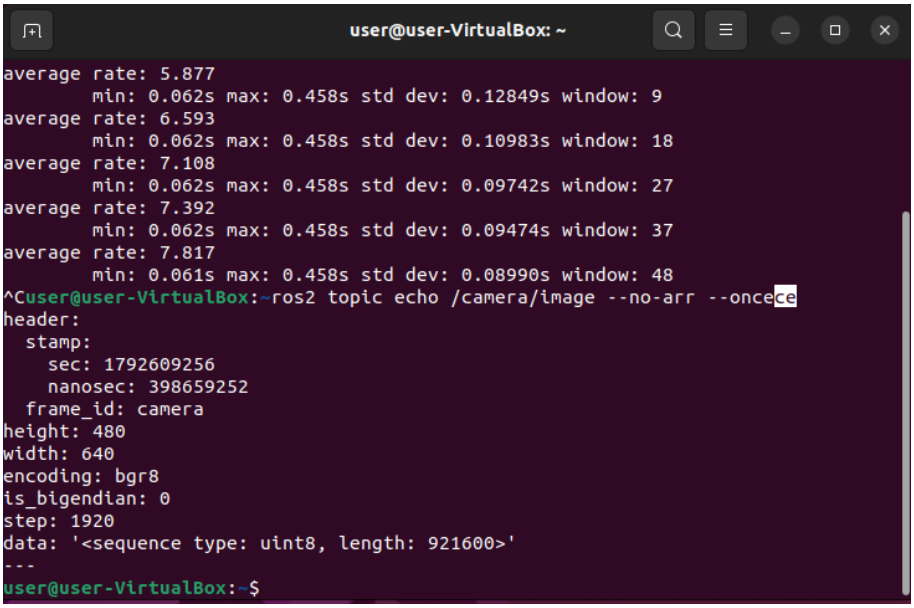


Рисунок 2 - Топік камери /camera/image

Надходження відео підтверджено переглядом у rqt_image_view (рис. 3).

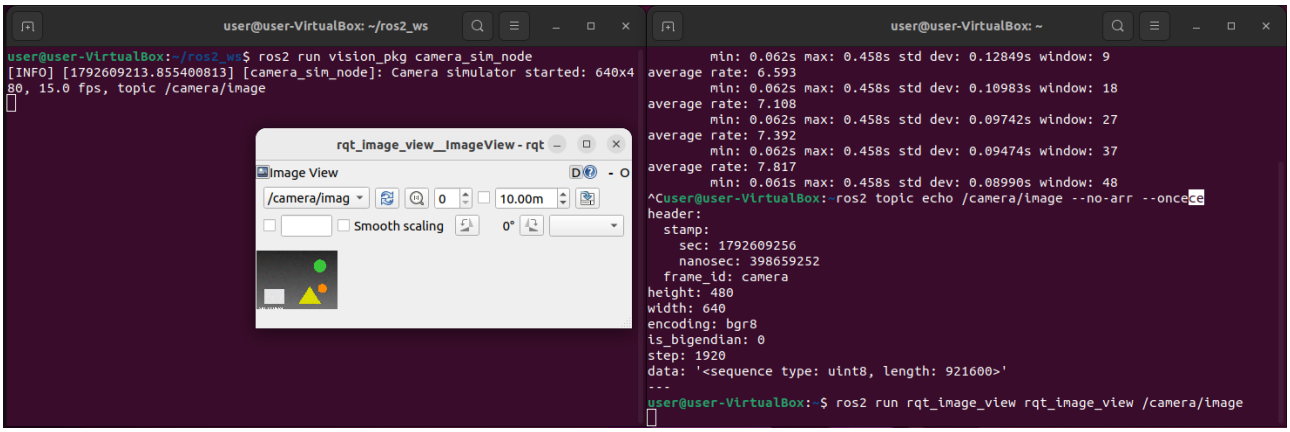


Рисунок 3 - Відеопотік із топіка /camera/image

Оскільки Vision Node отримує зображення лише з топіка /camera/image, симулятор можна замінити реальною камерою (наприклад, драйвером v4l2_camera з перенаправленням image_raw:=/camera/image) без жодних змін у коді обробки.

Vision Node

Нода vision_node реалізована класом VisionNode, успадкованим від rclpy.node.Node. Вона підписана на топік /camera/image, для кожного кадру виконується функція зворотного виклику image_callback, яка конвертує

повідомлення в масив OpenCV, виконує обробку, публікує результат і відображає його у вікні.

Алгоритм обробки

Кожен кадр проходить такий конвеєр:

1. **Градації сірого.** Три канали зводяться до одного (Y = 0.299R + 0.587G + 0.114B). Далі алгоритми працюють з однією яскравістю, що в 3 рази зменшує обсяг обчислень.
2. **Розмиття за Гауссом** з ядром 5x5. Підвищує стійкість наступних кроків до шуму камери: без нього поріг і детектор країв реагували б на окремі «шумові» пікселі.
3. **Порогова обробка** з порогом 127: пікселі яскравіші за поріг стають білими (255), решта чорними (0). Результат - маска яскравих об'єктів.
4. **Детекція країв Canny** (пороги 50 і 150). Вибирає лінії різкої зміни яскравості, тобто межі об'єктів.
5. **Детекція руху.** Для двох сусідніх розмитих кадрів обчислюється модуль різниці absdiff. Пікселі зі зміною яскравості понад 25 вважаються рухомими. Маска розширюється (dilate), щоб злити близькі області, потім у ній знаходяться зв'язні контури, області площею понад 500 пікселів обводяться рамкою. Нерухомі об'єкти та статичний шум різницю не дають.

Результати кожного етапу відображаються у вікні Vision (мозаїка 3x2, функція cv2.imshow()). У вікні клавіша s зберігає всі етапи у файли каталогу results/ (рис. 4).

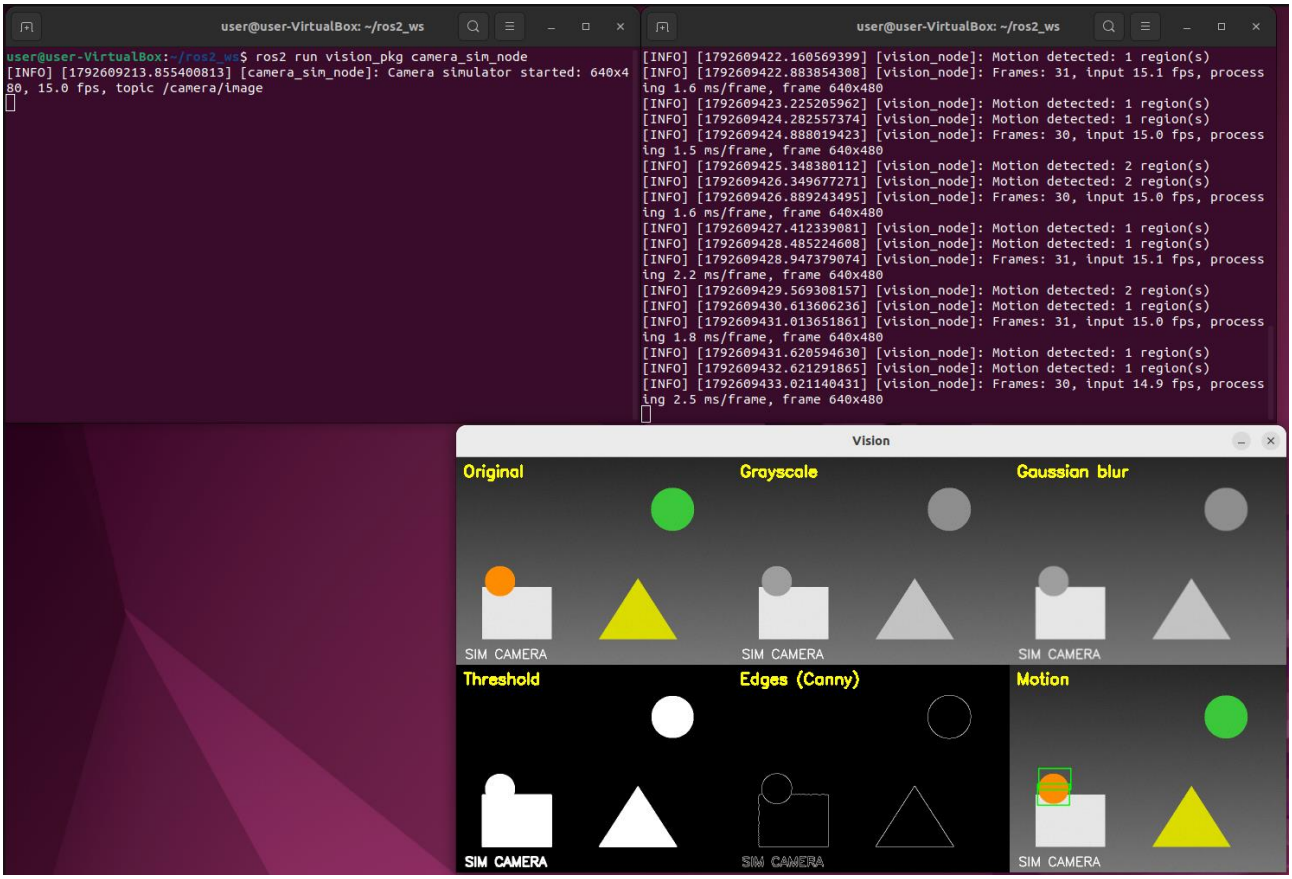


Рисунок 4 - Робота vision_node

Результати обробки

Кадри, збережені нодою, наведено на рис. 5-6. Оригінальне зображення (рис. 5) містить нерухомі фігури та рухоме помаранчеве коло.

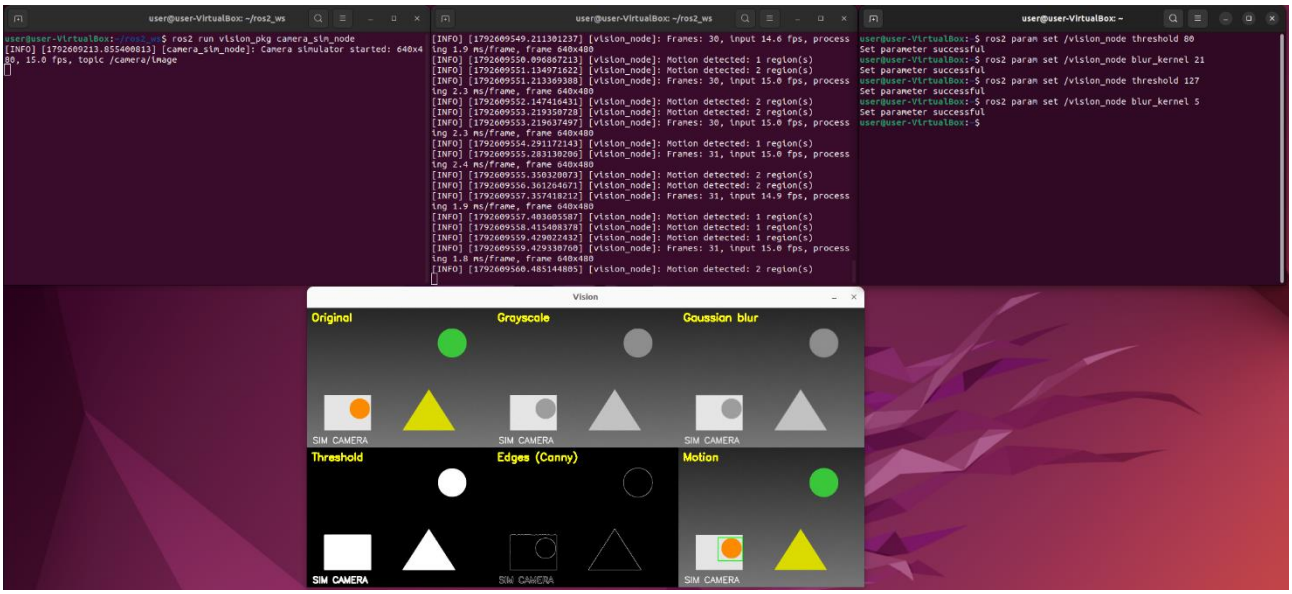


Рисунок 5 - Оригінальне зображення

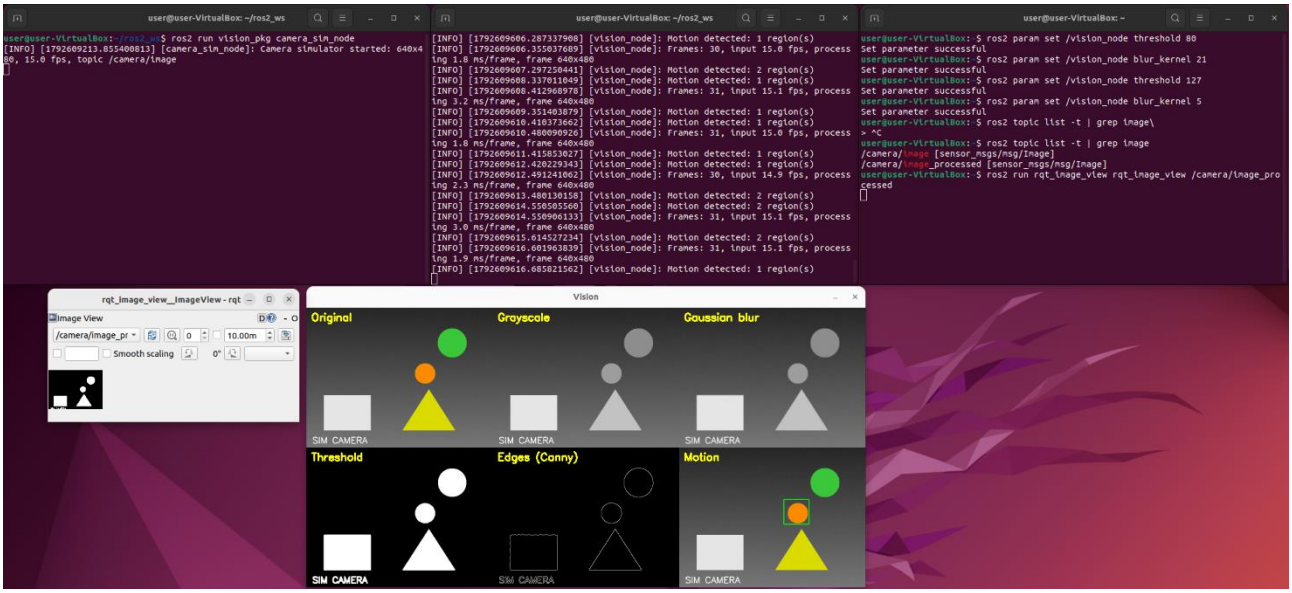


Рисунок 6 – Оброблене зображення

Таблиця 1 - Спостереження за етапами обробки

Етап	**Що видно на результаті**
Grayscale	Зображення без кольору, яскраві об'єкти світліші за фон
Gaussian blur	Краї й шум розмиті, дрібні деталі зникли
Threshold (127)	Білими залишилися об'єкти яскравіші за 127, фон чорний
Canny	Тонкі контури фігур, однорідні області порожні
Motion	Рамка лише навколо рухомого кола; нерухомі фігури без рамок

Інтеграція із системою ROS2

Оброблене (порогове) зображення публікується в топік /camera/image_processed (sensor_msgs/msg/Image, mono8). Його можна переглянути в rqt_image_view або використати іншими нодами.

Висновок

У ході лабораторної роботи розроблено пакет vision_pkg з Vision Node, яка підписується на топік камери /camera/image (sensor_msgs/msg/Image), перетворює кадри у формат OpenCV за допомогою cv_bridge і виконує базову обробку: перетворення в градації сірого, розмиття за Гауссом і порогову обробку. Додатково реалізовано детекцію країв та руху. Оригінальне та оброблені зображення відображаються у вікні cv2.imshow(), а результат порогової обробки публікується в окремий топік, тож його можуть використовувати інші ноди системи. Через відсутність фізичної камери

відеопотік імітує нода camera_sim_node, завдяки тому, що Vision Node залежить тільки від топіка, симулятор можна замінити реальною камерою без змін у коді.